

UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y
NATURALES



PROGRAMACIÓN CONCURRENTE - Año 2024

Trabajo Práctico N° 2:
Sistema de Agencia de Viajes

Profesores

- Ventre, Luis Orlando
- Ludemann, Mauricio

Integrantes

- BERNARDI, Mateo
- LEDESMA, Ignacio
- MADRID, Santiago

ÍNDICE

ÍNDICE.....	1
DESARROLLO.....	3
Análisis de la Red de Petri.....	3
Propiedades de la Red de Petri.....	3
Descripción de los Estados y Eventos de la PN.....	5
Invariantes.....	7
Invariantes de Transición (T-Invariants).....	7
Invariantes de Plaza (P-Invariants).....	8
Determinación de Hilos.....	10
Máxima Cantidad de Hilos simultáneos.....	10
1. Identificación de Invariantes de Transición (IT) y sus Plazas.....	10
2. Determinación de Plazas de Acción (PA).....	10
3. Análisis de Marcados y Suma Máxima.....	11
Cantidad de Hilos Necesarios.....	12
Caso 1: Gestión de Conflictos (Forks en P3 y P9).....	12
Caso 2: Gestión de Uniones (Join en P14).....	13
Conclusiones sobre la Cantidad de Hilos.....	13
Implementación de los Hilos.....	14
Políticas.....	15
Conflicto 1: Asignación de Vendedores.....	15
Conflicto 2: Resolución de Caso.....	16
Interacción entre el Monitor y la Política.....	16
Tiempos.....	18
Configuración de Tiempos Elegida.....	18
Análisis de Cotas Temporales.....	19
Límite Inferior Absoluto (Ley de Amdahl).....	19
Límite Superior Técnico (Escenario Secuencial).....	19
Intervalo de Convergencia.....	20
Análisis Práctico.....	20
Análisis de Sensibilidad: ¿Qué Sucede si Cambian los Tiempos?.....	22
A. Entrada Lenta ($T_1 = 200$ ms).....	22
B. Entrada Rápida ($T_1 = 5$ ms).....	22
C. Vendedores Lentos ($T_4 = T_5 = 200$ ms).....	23
D. Vendedores Rápidos ($T_4 = T_5 = 5$ ms).....	23
E. Vendedores con Tiempos Distintos.....	24
F. Confirmación Lenta ($T_9 = T_{10} = 200$ ms).....	24

G. Confirmación Rápida ($T_9 = T_{10} = 5 \text{ ms}$).....	25
H. Cancelación Lenta ($T_8 = 200 \text{ ms}$).....	25
I. Cancelación Rápida ($T_8 = 5 \text{ ms}$).....	26
Lectura de los Resultados.....	26
Conclusiones:.....	28
Cumplimiento de Invariantes.....	29
Invariantes de Plaza.....	29
Invariantes de Transición.....	30
Expresión Regular (RegEx) de la Red.....	30
Análisis de los Logs.....	33
Caso A: Análisis de la Política Balanceada.....	33
Caso B: Análisis de la Política Priorizada.....	34
CONCLUSIONES.....	35
REFERENCIAS BIBLIOGRÁFICAS.....	36
ANEXO A: TABLA DE MARCADOS ALCANZABLES (MAi) PARA LAS PA.....	37

DESARROLLO

Análisis de la Red de Petri

El primer acercamiento para este trabajo fue implementar la Red de Petri propuesta por la cátedra a través de la herramienta PIPE v4.3.2.

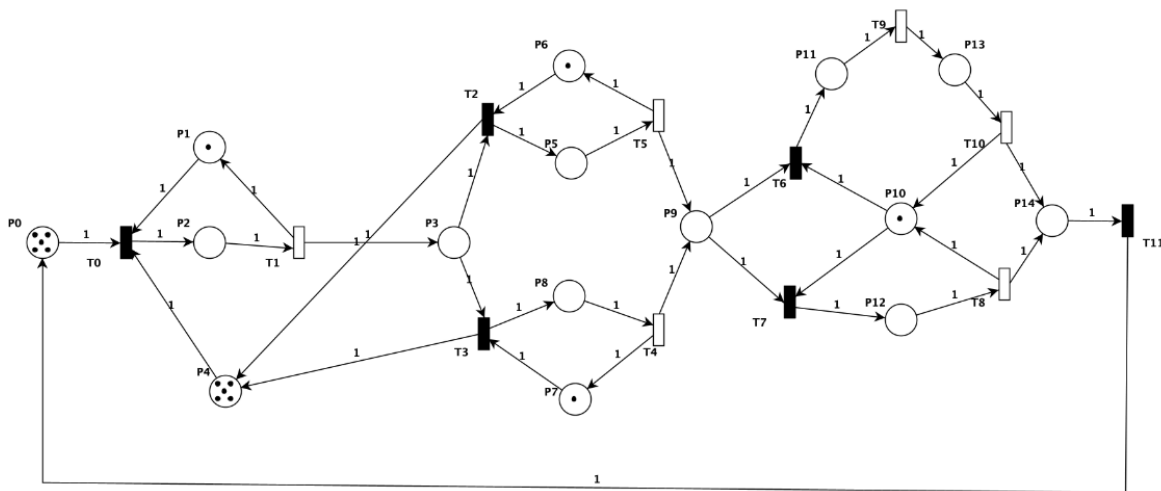


Figura 1: PN diseñada en PIPE

Esta herramienta no solamente nos permite ver el flujo de tokens a lo largo de la PN, sino que también realiza una análisis de la misma en torno a diferentes aspectos. A continuación se detallan algunos de ellos:

Propiedades de la Red de Petri

Petri net state space analysis results

Bounded	true
Safe	false
Deadlock	false

Figura 2: Resultados del Análisis del Espacio de Estados arrojados por PIPE

En base al análisis obtenido, podemos entender mejor el comportamiento de nuestro sistema:

1. **Bounded (Acotada):** significa que el número de tokens en **cualquier plaza** de la red NUNCA va a superar un valor máximo k o, dicho de otra forma, los tokens nunca van a crecer indefinidamente, lo que significa que el universo de estados es finito.

2. **Safe (Segura):** una red es segura cuando el límite de tokens en **todas las plazas** es exactamente 1. Como en el reporte es falso, significa que existe al menos una plaza en la red que puede acumular 2 o más tokens simultáneamente.
 - En la *Figura 1* se puede observar a simple vista que las plazas P0 y P4 ya empiezan con más de 1 token, así que aportan al resultado obtenido de que la red no es segura.
3. **Deadlock (Bloqueo Mutuo):** que este resultado haya dado falso significa que la red está libre de interbloqueos. Desde cualquier estado alcanzable, **siempre existirá al menos una transición que puede dispararse**, por lo que el sistema nunca se quedará congelado de forma permanente. Si hubiera sido verdadero, significa que para algún marcado ya no se puede volver a disparar ninguna de las transiciones y la red llegó a un estado sumidero.
4. **Liveness (Vivacidad):** para que una Red de Petri sea viva, se debe garantizar que, desde cualquier estado (marcado) alcanzable, siempre sea posible volver a activar y disparar absolutamente todas las transiciones de la red, sin que **ninguna transición quede inutilizable de forma permanente**. Si bien PIPE no arroja un resultado sobre esta propiedad, se puede deducir de la siguiente forma:
 - Analizando la *Figura 1*:
 - Debido a la transición T11 que va de vuelta a la plaza P0, los tokens no tienen un punto final o sumidero donde desaparecer, así que están **obligados a seguir de forma indefinida**.
 - Las decisiones o conflictos de la red (bifurcaciones en P3 hacia T2 o T3, o en P9 hacia T6 o T7) están controladas por plazas particulares que ya cuentan con un token (P6, P7 y P10). La estructura de la red garantiza que cuando los tokens de estas plazas de control se consumen por el disparo de una u otra transición, esos mismos tokens son devueltos exactamente a la plaza de la que vinieron. Esto hace que sea **imposible que un recurso de sincronización se pierda**.
 - Al combinar estos aspectos con un espacio de estados finito (red acotada) y la ausencia de interbloqueos, el flujo del sistema progresa eternamente. Debido a que la estructura del grafo impide la exclusión permanente de cualquier ruta o transición, la posibilidad de disparo para cada componente de la red se mantiene latente de forma indefinida. Esto constituye la prueba de que **la red es viva**.

Descripción de los Estados y Eventos de la PN

Antes de seguir avanzando con el análisis de la Red de Petri, consideramos apropiado describir qué aspecto del modelo representan las plazas y las transiciones. Para ello, realizamos las siguientes tablas:

PLAZA	TIPO DE ENTIDAD	DESCRIPCIÓN
P0	Estado de Proceso	Generador de Clientes: buffer inicial de clientes listos para ingresar a la agencia
P1	Recurso Compartido	Puerta: Mutex para regular el ingreso unitario a recepción
P2	Estado de Proceso	Cliente ingresando
P3	Estado de Proceso	Cola de Clientes esperando a ser atendidos por algún vendedor
P4	Recurso Compartido	Capacidad de la Sala de Espera: restringe el aforo máximo de la sala
P5 y P8	Estados de Proceso	Cliente siendo atendido por Vendedor de la P6 o P7 respectivamente
P6 y P7	Recurso Compartido	Disponibilidad del Vendedor 1 o 2
P9	Estado de Proceso	Cola de Espera de Caja: caso (confirmación o cancelación de reserva) en espera de ser atendido y procesado
P10	Recurso Compartido	Recurso Caja: garantiza la exclusión mutua durante el procesamiento de un caso
P11	Estado de Proceso	Reserva aprobada
P12	Estado de Proceso	Reserva cancelada
P13	Estado de Proceso	Cliente en fila de pago
P14	Estado de Proceso	Cola de salida (Check-Out)

Tabla 1: Tabla Plazas-Estados de la PN

- **Estado de Proceso / Transitorio (¿Dónde está la entidad?):** Es una etapa transitoria. Los tokens que se encuentran en estas plazas representan unidades de trabajo activas que están esperando o recibiendo un servicio.
- **Recurso de Control / Compartido (¿Qué se necesita para avanzar?):** Son semáforos, variables de exclusión mutua (mutex) o buffers físicos que limitan el paso. Representan las herramientas o restricciones del entorno necesarias para seguir avanzando.

TRANSICIÓN	DESCRIPCIÓN
T0	Ingreso de Clientes: un cliente sale del buffer inicial e ingresa a la agencia
T1	El cliente pasa a la sala de espera de la agencia
T2 y T3	Cliente es atendido por el Vendedor de P6 o P7 respectivamente
T4 y T5	Cliente terminó de ser atendido
T6	Se procede con la confirmación de la reserva, tomando el recurso “caja”
T7	Se procede con la cancelación de la reserva, tomando el recurso “caja”
T8	Se procesa efectivamente la baja de la reserva, liberando el recurso “caja”
T9	El sistema confirma formalmente la reserva, derivando al cliente a la instancia de pago en efectivo
T10	Cliente abona la confirmación de la reserva, liberando el recurso “caja”
T11	Egreso de Clientes: Cliente realiza el check-out y abandona la agencia

Tabla 2: Tabla Transiciones-Eventos de la PN

Invariantes

En Redes de Petri, un invariante es una propiedad estructural y matemática del grafo que se mantiene constante independientemente de los disparos de transiciones que ocurran en el sistema o del estado en el que se encuentre. PIPE identificó los siguientes invariantes en la red:

Petri net invariant analysis results

T-Invariants

T0	T1	T10	T11	T2	T3	T4	T5	T6	T7	T8	T9
1	1	0	1	0	1	1	0	0	1	1	0
1	1	1	1	0	1	1	0	1	0	0	1
1	1	0	1	1	0	0	1	0	1	1	0
1	1	1	1	1	0	0	1	1	0	0	1

The net is covered by positive T-Invariants, therefore it might be bounded and live.

P-Invariants

P0	P1	P10	P11	P12	P13	P14	P2	P3	P4	P5	P6	P7	P8	P9
0	1	0	0	0	0	0	1	0	0	0	0	0	0	0
0	0	1	1	1	1	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	0	1	0	0	1	1
0	0	0	0	0	0	0	1	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	1	1	0

The net is covered by positive P-Invariants, therefore it is bounded.

P-Invariant equations

$$M(P1) + M(P2) = 1$$

$$M(P10) + M(P11) + M(P12) + M(P13) = 1$$

$$M(P0) + M(P11) + M(P12) + M(P13) + M(P14) + M(P2) + M(P3) + M(P5) + M(P8) + M(P9) = 5$$

$$M(P2) + M(P3) + M(P4) = 5$$

$$M(P5) + M(P6) = 1$$

$$M(P7) + M(P8) = 1$$

Figura 3: Invariantes de la PN arrojados por PIPE

Invariantes de Transición (T-Invariants)

Un invariante de transición define una secuencia de disparos que devuelve a la red exactamente al mismo marcado desde el cual comenzó.

El análisis arrojado por PIPE expone 4 invariantes de transición. Cada uno de ellos representa a un cliente completando un circuito de negocio específico desde que ingresa hasta que se retira del local.

- **T-Invariante 1 (Camino Inferior por Cancelación):** [T0, T1, T3, T4, T7, T8, T11]
 - Representa el flujo de un cliente que ingresa al local, pasa a la sala de espera y es derivado con el Vendedor 2 (P7). Al finalizar la atención, el recurso de caja deriva el

trámite al flujo operativo de cancelación de la reserva. Tras hacerse efectiva la baja, el cliente realiza el check-out y se retira de la agencia.

- **T-Invariante 2 (Camino Inferior de Venta Exitosa):** [T0, T1, T3, T4, T6, T9, T10, T11]
 - Representa el flujo en el cual el cliente es atendido por el Vendedor 2 (P7), pero en la instancia de caja el trámite avanza de manera conforme (T6). El sistema confirma formalmente la reserva (T9), el cliente abona efectivamente en la fila de pago (T10) y finalmente realiza su salida del local.
- **T-Invariante 3 (Camino Superior por Cancelación):** [T0, T1, T2, T5, T7, T8, T11]
 - Es un recorrido análogo al Invariante 1, con la diferencia estructural de que el cliente es atendido en la rama comercial superior por el Vendedor 1 (P6), activando la secuencia de transiciones T2 y T5. Finaliza con la cancelación en el recurso de caja.
- **T-Invariante 4 (Camino Superior de Venta Exitosa):** [T0, T1, T2, T5, T6, T9, T10, T11]
 - Simboliza el proceso de confirmación de reserva del cliente a través del puesto de atención del Vendedor 1 (P6).

Invariantes de Plaza (P-Invariants)

Una invariante de plaza es una ecuación lineal que demuestra que la suma ponderada de tokens en un conjunto determinado de plazas permanece constante para cualquier marcado alcanzable.

El análisis arrojado por PIPE expone 6 invariantes de plaza:

- **P-Invariante 1 (Mutex del ingreso):** $M(P1) + M(P2) = 1$
 - La plaza P1 actúa como el semáforo binario de control de entrada (la puerta). Al estar igualado a 1, garantiza estrictamente la exclusión mutua en la recepción, es decir, solo puede haber un único cliente ingresando a la vez.
- **P-Invariante 2 (Caja individual):** $M(P10) + M(P11) + M(P12) + M(P13) = 1$
 - Este invariante está gobernado por el Recurso Caja (P10), el cual funciona como un Mutex. Asegura que las etapas de confirmación en P11 y P13 y el procesamiento de la cancelación en P12 sean mutuamente excluyentes entre sí y que no puede haber más de un cliente a la vez siendo atendido en la caja.
- **P-Invariante 3 (Conservación de Clientes en el Sistema):**

$$M(P0) + M(P2) + M(P3) + M(P5) + M(P8) + M(P9) + M(P11) + M(P12) + M(P13) + M(P14) = 5$$
 - Esta invariante global demuestra que la cantidad total de tokens distribuidas a lo largo de todas las etapas activas del proceso es estrictamente constante e igual a 5. Los clientes dentro del sistema siempre son 5, nunca se “pierden” ni se crean de la nada, simplemente cambian de estado en una red cerrada.
- **P-Invariante 4 (Sala de recepción):** $M(P2) + M(P3) + M(P4) = 5$
 - Este invariante modela el control del Aforo Físico de la Sala de Recepción. La plaza P4 representa las sillas libres disponibles en la sala. La ecuación asegura de forma constante que la suma de clientes entrando a la Agencia, los clientes esperando en P3 y los asientos vacíos sea siempre igual a 5.

- **P-Invariante 5 (Vendedor 1):** $M(P5) + M(P6) = 1$
 - Esta invariante modela la disponibilidad del Vendedor 1 (P6). Al ser un recurso unitario, fuerza a que el puesto de atención de la rama superior opere en exclusión mutua. Osea, o el vendedor está libre (P6 con 1 token) o atendiendo exactamente a un solo cliente (P5 con 1 token)
- **P-Invariante 6 (Vendedor 2):** $M(P7) + M(P8) = 1$
 - Modela de forma idéntica a la ecuación anterior la disponibilidad del Vendedor 2 (P7).

Determinación de Hilos

Para abordar esta sección, vamos a estructurar el análisis combinando el marco teórico formal provisto por el paper^[1] otorgado por la cátedra con las restricciones algorítmicas de paralelismo exigidas en el enunciado, a saber:

1. Si el invariante de transición tiene un conflicto con otro invariante, debe haber un hilo encargado de la ejecución de la/s transición/es anterior/es al conflicto y luego un hilo por invariante
2. Si el invariante de transición presenta un join con otro invariante de transición, luego del join debe haber tantos hilos como tokens simultáneos en la plaza encargados de las transiciones restantes dado que hay un solo camino.

Máxima Cantidad de Hilos simultáneos

Siguiendo estrictamente la metodología del artículo de investigación^[1]:

1. Identificación de Invariantes de Transición (IT) y sus Plazas

Como vimos anteriormente, la red cuenta con 4 invariantes de transición que describen los ciclos del software:

- $IT_1 = \{T0, T1, T3, T4, T7, T8, T11\}$ (Vendedor 2 - Cancelación)
- $IT_2 = \{T0, T1, T3, T4, T6, T9, T10, T11\}$ (Vendedor 2 - Confirmación)
- $IT_3 = \{T0, T1, T2, T5, T7, T8, T11\}$ (Vendedor 1 - Cancelación)
- $IT_4 = \{T0, T1, T2, T5, T6, T9, T10, T11\}$ (Vendedor 1 - Confirmación)

Las plazas relacionadas a cada IT son:

- $PIT_1 = \{P0, P1, P2, P3, P4, P7, P8, P9, P10, P12, P14\}$
- $PIT_2 = \{P0, P1, P2, P3, P4, P7, P8, P9, P10, P11, P13, P14\}$
- $PIT_3 = \{P0, P1, P2, P3, P4, P5, P6, P9, P10, P12, P14\}$
- $PIT_4 = \{P0, P1, P2, P3, P4, P5, P6, P9, P10, P11, P13, P14\}$

2. Determinación de Plazas de Acción (PA)

Aplicando la ecuación formal del paper^[1]:

$$PA_i = PIT_i - \{PIT_{restricciones} \cup PIT_{recursos} \cup PIT_{idle}\}$$

- $PIT_{restricciones}$: Son plazas de control adicionales introducidas explícitamente para limitar el aforo o evitar condiciones catastróficas como el Deadlock. Modelan la capacidad máxima de un entorno físico o virtual

- $PIT_{recursos}$: Son plazas que modelan dispositivos o entidades físicas que son compartidos de forma competitiva por los procesos concurrentes. Un proceso adquiere el recurso para trabajar y lo libera al terminar.
- PIT_{idle} : Son todas aquellas plazas que representan estados de reposo, buffers de entrada o puntos de partida y llegada de los procesos en un ciclo cerrado. También se conocen como plazas ociosas o de espera inicial.
- PA_i : Representan el conjunto de estados operacionales puros por los que transita una entidad dentro de un invariante de transición determinado.

Al remover los recursos físicos ($P1$, $P6$, $P7$ y $P10$) y de control ($P4$) y las plazas idle/restricción ($P0$), obtenemos las plazas relacionadas a acciones de cada IT:

- $PA_1 = \{P2, P3, P8, P12, P14\}$
- $PA_2 = \{P2, P3, P8, P11, P13, P14\}$
- $PA_3 = \{P2, P3, P5, P12, P14\}$
- $PA_4 = \{P2, P3, P5, P11, P13, P14\}$

De esta forma, el universo de Plazas de Acción (donde los hilos ejecutan tareas operativas) queda definido como:

$$PA = \{P2, P3, P5, P8, P9, P11, P12, P13, P14\}$$

3. Análisis de Marcados y Suma Máxima

En PIPE podemos generar una Tabla de Marcados Alcanzables a partir de la Red de Petri modelada. Esta tabla enumera de forma exhaustiva los 65 estados únicos e irrepetibles del sistema (desde M_0 hasta M_{64}) y representa la conversión directa a filas y columnas de los nodos del Grafo de Alcanzabilidad.

De esta tabla podemos extraer el universo completo de vectores de marcado posibles para las Plazas de Acción determinadas anteriormente, al cual llamaremos MA , que básicamente nosotros representamos como otra tabla pero con sus columnas limitadas solamente a las Plazas de Acción (ver [Anexo A: Tabla de Marcados Alcanzables para las Plazas de Acción](#)). A partir de esta tabla, realizamos el siguiente análisis:

- En cada marcado posible realizamos la suma de las marcas e identificamos cual es la mayor suma posible o, dicho de otra forma, el “marcado máximo”.
- El artículo^[4] define formalmente los hilos como los agentes encargados de ejecutar el modelo mediante vectores de marcado en las plazas de acción, que se puede interpretar como que un *token* ubicado en una plaza de acción representa un hilo ejecutando una tarea. En consecuencia, la suma instantánea de tokens en dichas plazas equivale exactamente al número de hilos operando en paralelo en ese momento del sistema

De esta forma, el marcado máximo mapea el pico más alto de concurrencia real que la estructura de la red tolera. Por lo tanto, la **cantidad máxima de hilos activos simultáneos va a ser 5**.

Cantidad de Hilos Necesarios

Para segmentar las responsabilidades del software y maximizar el paralelismo, aplicamos las dos directivas del enunciado basadas en la estructura de los invariantes:

Caso 1: Gestión de Conflictos (Forks en P3 y P9)

El enunciado establece que ante un conflicto estructural, debe asignarse un hilo encargado de la ejecución de las transiciones anteriores y, posteriormente, hilos independientes por cada rama del invariante.

1. Conflicto en la Sala de Espera (P3):

- Las transiciones previas al conflicto son $T0$ (Ingreso al Local) y $T1$ (Ingreso a la Sala de Espera). Se asigna un **Hilo Generador encargado de activar el arribo de clientes desde el pool inicial ($P0 \rightarrow P2$)**.
- **Resolución del Conflicto en P3:** El flujo se bifurca hacia el Vendedor 1 ($T2$) o el Vendedor 2 ($T3$). Se asigna un **hilo para la rama superior (IT_3, IT_4)** y un **hilo para la rama inferior (IT_1, IT_2)**. Más adelante se resolverá la elección de atención a través de una Política.
- **Realización de Tareas (Atención de Clientes):** Tras la resolución del conflicto en la sala de espera, el flujo entra en tramos independientes. Para maximizar el paralelismo, se hace una distinción entre los **hilos únicamente encargados de resolver el conflicto** (los 2 del punto anterior) y los **hilos destinados a realizar la tarea operativa (la atención de un cliente)**:
 - **Línea Vendedor 1 (Tramo Superior):** Se asigna un **hilo responsable de la secuencia $T2 \rightarrow T5$** , donde el estado de atención se representa con un token en la plaza $P5$
 - **Línea Vendedor 2 (Tramo Inferior):** Se asigna un **hilo responsable de la secuencia $T3 \rightarrow T4$** , donde el estado de atención se representa con un token en la plaza $P8$

Esto garantiza que en el software ambos agentes procesan solicitudes simultáneamente sin interferirse.

2. Conflicto en la Resolución de Caso (P9):

- Las transiciones previas ($T4$ y $T5$) ya están cubiertas por los hilos de los vendedores
- En $P9$ ocurre el segundo conflicto estructural de la red regulado por el Recurso Caja ($P10$) para ir hacia la Confirmación ($T6$) o Cancelación ($T7$). Se asigna un **hilo para la rama de Confirmación (IT_2, IT_4)** y **otro para la rama de Cancelación (IT_1, IT_3)**.

- Dado que el **recurso en P10 es compartido por las invariantes** (en contraposición al caso anterior), se opera bajo una exclusión mutua binaria, por lo que las rutas que parten de la bifurcación son mutuamente excluyentes en el tiempo. Esta propiedad de la red permite que los hilos encargados de las transiciones en conflicto ($T6$ y $T7$) continúen la ejecución de sus tareas ($T8$ o $T9 \rightarrow T10$) de manera segura. De este modo, no es necesario diferenciar los hilos que resuelven el conflicto de los que realizan la tarea operativa, logrando una implementación más eficiente.

Caso 2: Gestión de Uniones (Join en P14)

El enunciado indica que tras un join, debe haber tantos hilos como tokens simultáneos permita la plaza resultante.

- Las ramas de Pago ($T10$) y Cancelación ($T8$) convergen en la plaza previa a la salida $P14$.
- Aunque el sistema maneja un flujo total de 5 clientes concurrentes, el acceso a las ramas que alimentan a $P14$ está estrictamente regulado por la exclusión mutua del Recurso Caja ($P10$). Como $P10$ posee un marcado máximo de 1 token, es matemáticamente imposible que dos procesos depositen tokens a la vez en $P14$. Por ende se asigna un único **hilo encargado de ejecutar la transición de salida final $T11$ para reinsertar el token en $P0$** .

Conclusiones sobre la Cantidad de Hilos

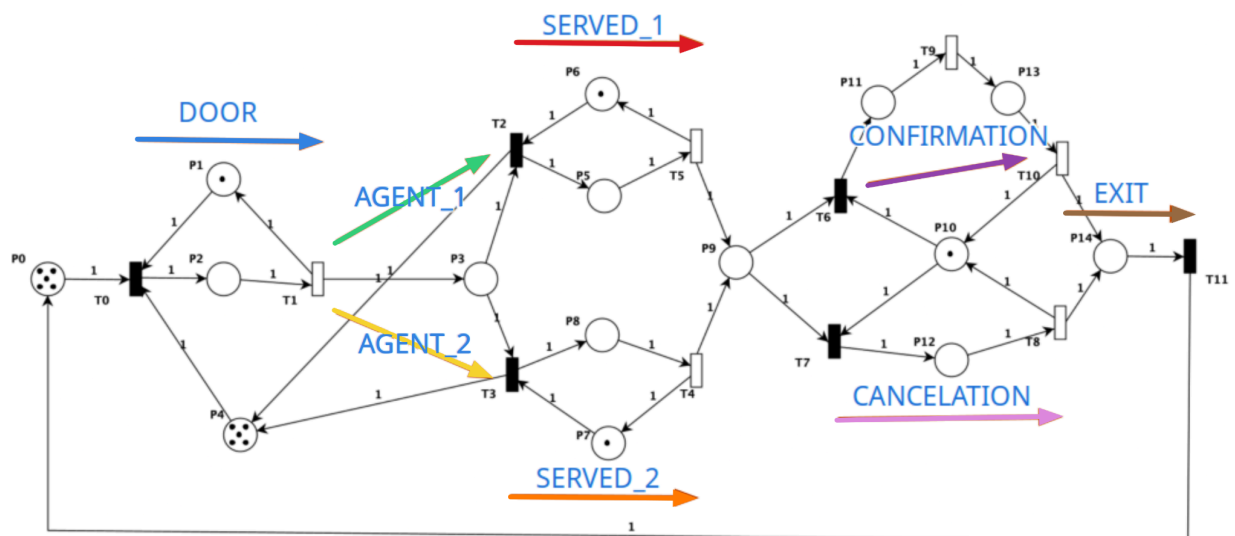


Figura 4: Responsabilidades de los Hilos de la PN

De esta forma, podemos concluir que el programa estará compuesto por un **total de 8 hilos** con responsabilidades bien delimitadas en distintos segmentos del programa, y de esos 8 hilos habrá un **máximo de 5 siendo ejecutados simultáneamente**.

Implementación de los Hilos

Se optó por heredar de la clase Thread, lo que simplifica significativamente la instanciación y gestión de los hilos, ya que cada entidad del sistema (la puerta, los agentes, la confirmación, la cancelación y la salida) se corresponde naturalmente con un proceso independiente que debe ejecutarse de forma autónoma. Al heredar de Thread, cada clase del paquete tasks encapsula su propio ciclo de vida y su método run() describe la lógica de negocio que esa entidad ejecuta de manera ininterrumpida mientras el sistema esté activo.

Por otro lado, la decisión de NO hacer a Monitor ni a PetriNet clases vivas responde a una cuestión de diseño deliberada:

- El monitor, en particular, actúa como un mediador central que arbitra el acceso concurrente a la Red de Petri mediante un semáforo (mutex) y una cola para despertar los hilos bloqueados cuando una transición que ellos esperaban se habilita. Hacerlo un hilo más rompe el esquema donde los hilos vivos se limitan a ser productores de disparos y el monitor el árbitro pasivo. Además, por definición, el monitor es una estructura pasiva que no tiene vida ni hilos propios.
- PetriNet es una estructura de datos que debe ser consultada y modificada atómicamente; convertirla en hilo le daría un comportamiento activo que no se condice con su rol de limitarse a describir la estructura de negocio.

Cada tarea conoce únicamente el índice de la transición (o conjunto de transiciones) que le corresponde disparar, pero NO sabe ni le importa que otras tareas existen ni como se resuelven sus conflictos. Toda la lógica del negocio queda encapsulada en CL_Policy, que es de lo que hablaremos en la siguiente sección.

Finalmente, cabe mencionar que la elección de heredar de Thread por sobre implementar Runnable se justifica por dos motivos concretos:

- Cada entidad del modelo tiene identidad propia y ciclo de vida acoplado al de la simulación: arracan con thread.start() en Main y terminan cuando su condición de corte se cumple (por ejemplo, DoorTask termina cuando receivedClients alcanza totalClients).
- Nombrar cada hilo resulta muy conveniente para el modo debug, ya que los logs y mensajes de error del monitor incluyen Thread.currentThread.getName(), lo que permite identificar qué entidad disparó cada transición. Si bien esto podría lograrse con Runnable y new Thread(run, name), se optó por extender Thread para que la autoidentificación del hilo quede encapsulada dentro de la clase.

Políticas

En una Red de Petri, cuando dos o más transiciones comparten una misma plaza de entrada y compiten simultáneamente por el consumo de un mismo token, se produce un conflicto. Al trasladar el modelo a una implementación en software real, es vital dotar al sistema de una lógica de negocio controlada, y para ello se introducen las Políticas.

Para administrar centralizadamente cuál de los caminos lógicos de la Red debe progresar cada vez que el Monitor detecta transiciones sensibilizadas compitiendo por un recurso, se implementó la clase CL_Policy. Como se solicitó, el sistema contempla dos perfiles a través de la flag `balanced`:

1. **Modo Balanceado (Equidad de Carga):** Distribuye equitativamente los procesos de manera dinámica en base a las estadísticas del sistema (agentes atienden igual cantidad de clientes y hay tantas reservas confirmadas como canceladas)
2. **Modo No Balanceado (Priorización por Umbral):** Fuerza un sesgo estadístico en el comportamiento a largo plazo del sistema mediante tasas fijas configurables (la cátedra solicita específicamente que el 75% de las reservas sean administradas por el agente en P6 y el 25% para el agente en P7, y que al finalizar el programa un 80% del total de las reservas deben haber sido confirmadas y un 20% canceladas)

Conflicto 1: Asignación de Vendedores

- **Origen de la Competencia:** Los tokens en la Sala de Espera (P3) compiten por ser atendidos en la rama del Vendedor 1 (T2) o del Vendedor 2 (T3).

```
private Integer resolveP6n7Conflict(int[] feasibleTransitions) {
    if (feasibleTransitions[TRANSITION_P6] != 1 || feasibleTransitions[TRANSITION_P7] != 1) {
        return null; // No hay conflicto
    }

    if (balanced) {
        return agent1.getCustomersP6() < agent2.getCustomersP7() ? TRANSITION_P6 : TRANSITION_P7;
    }

    int totalCustomers = agent1.getCustomersP6() + agent2.getCustomersP7();
    return agent1.getCustomersP6() <= totalCustomers * P6_PRIORITY_THRESHOLD
        ? TRANSITION_P6
        : TRANSITION_P7;
}
```

Figura 5: Resolución del Conflicto de Vendedores según la Política

- **Si `balanced = true`:** Se evalúan las métricas de los hilos y se dispara la transición del puesto que posea la menor carga de clientes atendidos en ese instante de tiempo (`agent1.getCustomersP6() < agent2.getCustomersP7()`)
- **Si `balanced = false`:** Se aplica la restricción estricta de la cátedra para romper la simetría. El algoritmo calcula continuamente la tasa de atención acumulada de la rama superior frente a la demanda total procesada por ambos agentes. Si el vendedor prioritario (P6) no ha excedido su cuota del 75% (`P6_PRIORITY_THRESHOLD = 0.75`), se fuerza el

disparo de T2 para forzar la derivación hacia el Vendedor 1, de lo contrario se balancea el flujo hacia T3.

Conflicto 2: Resolución de Caso

- **Origen de la Competencia:** Los casos en espera de resolución (P9) deben ser bifurcados hacia el canal de Confirmación (T6) o de Cancelación (T7).

```
private Integer resolveP11n12Conflict(int[] feasibleTransitions) {
    if (feasibleTransitions[TRANSITION_P11] != 1 || feasibleTransitions[TRANSITION_P12] != 1) {
        return null;
    }

    if (balanced) {
        return confirmation.getConfirmations() < cancellation.getCancellations()
            ? TRANSITION_P11
            : TRANSITION_P12;
    }

    int totalDecisions = confirmation.getConfirmations() + cancellation.getCancellations();
    return confirmation.getConfirmations() <= totalDecisions * P11_PRIORITY_THRESHOLD
        ? TRANSITION_P11
        : TRANSITION_P12;
}
```

Figura 6: Resolución del Conflicto de Resolución de Casos según la Política

- **Si `balanced = true`:** El sistema balancea las estadísticas totales de resolución de los casos (`confirmation.getConfirmations() < cancellation.getCancellations()`).
- **Si `balanced = false`:** Se parametriza el comportamiento comercial para inyectar un 80% de Confirmaciones y un 20% de Cancelaciones (`P11_PRIORITY_THRESHOLD = 0.80`).

Interacción entre el Monitor y la Política

El Monitor es el encargado de proteger el estado de la Red de Petri y gestionar las colas de hilos bloqueados mediante un semáforo de exclusión mutua general. En este contexto, la Política es un componente desacoplado que solo interviene cuando el sistema requiere tomar una decisión de negocio ante un conflicto efectivo.

Cuando un hilo dispara de manera exitosa una transición a través del método `fireTransitionAndUpdateState(transition)`, las condiciones de la red cambian. Luego el hilo en ejecución asume la responsabilidad de revisar si existen otros hilos bloqueados que ahora puedan progresar:

```

private boolean hasWaitingThreads() {
    int[] enabledTransitions = rdp.getEnabledTransitions();
    int[] waitingTransitions = queue.getWaitingTransitions();

    for (int i = 0; i < TRANSITIONS; i++) {
        if (enabledTransitions[i] * waitingTransitions[i] == 1) {
            return true; // Transición habilitada Y en cola de espera
        }
    }
    return false;
}

private void wakeUpWaitingThreads() {
    int[] enabledTransitions = rdp.getEnabledTransitions();
    int[] waitingTransitions = queue.getWaitingTransitions();
    int[] feasibleTransitions = new int[TRANSITIONS];

    for (int i = 0; i < TRANSITIONS; i++) {
        // Cuales transiciones están habilitadas Y en cola de espera?
        feasibleTransitions[i] = enabledTransitions[i] * waitingTransitions[i];
    }

    int nextTransition = policy.selectTransition(feasibleTransitions);
    queue.releaseTransition(nextTransition);
    debugLog("Hilo " + Thread.currentThread().getName() + " ha despertado T" + nextTransition);
}

```

Figura 7: Monitor delegando a la Política la decisión de Transición durante un conflicto

Para ello, el Monitor invoca los métodos `hasWaitingThreads()` y `wakeUpWaitingThreads()`, obteniendo un vector `feasibleTransitions` (Transiciones Factibles), el cual aísla e identifica únicamente aquellas transiciones que están habilitadas por el estado actual de la PN y, al mismo tiempo, tienen al menos un hilo esperando en la cola de espera del monitor (`CL_Queue`).

Una vez generado dicho vector, el Monitor delega la responsabilidad de la selección al método principal de la política con `policy.selectTransition(feasibleTransitions)`. Cuando la política retorna el índice de la transición óptima (`nextTransition`), el Monitor ejecuta `queue.releaseTransition(nextTransition)` para que el hilo actual despierte al hilo bloqueado encargado de esa transición específica.

De esta forma, el Monitor desconoce por completo las reglas comerciales de la agencia de viajes; solo entiende de redes de petri, tokens, plazas y bloqueos. La lógica de negocio está totalmente encapsulada en `CL_Policy`. □

Tiempos

Una vez implementado el Monitor y el sistema funcionando, ya podemos implementar transiciones temporales. Hasta aquí, los hilos compiten entre sí únicamente por la disponibilidad de tokens y por el orden en que el planificador de Java les asigna la CPU. En esta sección incorporamos una semántica de tiempo discreto sobre el conjunto de transiciones que modelan tareas humanas o de negocio, con el fin de aproximar la simulación al comportamiento que tendría nuestro modelo de Agencia de Viajes en la realidad.

La Red está compuesta por 12 transiciones. A los efectos del análisis temporal, conviene agruparlas en 2 clases bien diferenciadas según lo que el modelado representa:

- **Transiciones Operativas:** Modelan tareas humanas o de negocio con duración no despreciable. Son las únicas a las que se les asigna retardo. Son $\{T1, T4, T5, T8, T9, T10\}$.
- **Transiciones de Control:** Modelan la lógica del Monitor, no una tarea con duración propia. Su disparo representa un cambio de estado interno el cual se considera instantáneo. Son $\{T0, T2, T3, T6, T7, T11\}$.

Configuración de Tiempos Elegida

Cuando terminamos de implementar las políticas, notamos que la política balanceada se cumplía a la perfección pero la prioritaria, aún cuando la lógica era correcta, no otorgaba los resultados esperados. Tras investigar y empezar a avanzar con la implementación de las transiciones temporales, nos percatamos de que el tiempo tenía un impacto directo en los resultados para ambas políticas, lo que nos daba la pista de que probablemente si elegíamos el tiempo correcto, la política priorizada iba a cumplirse como esperábamos. Luego de diferentes cambios de alfa en las transiciones, logramos llegar a los resultados esperados para ambas políticas. El objetivo principal era que, sin importar cual sea la política, el tiempo total de ejecución sea similar y que los resultados sean los esperados para cada una.

La siguiente tabla resume los alfa utilizados en el código para ambas políticas

Transición	Plaza que consume	Política Balanceada	Política Priorizada
T1 (entrada a recepción)	P2	110 ms	110 ms
T4 (Vendedor 2)	P4	50 ms	50 ms
T5 (Vendedor 1)	P4	50 ms	50 ms
T8 (Cancelación)	P12	50 ms	25 ms
T9 (Confirmación)	P12	50 ms	50 ms
T10 (Pago)	P13	50 ms	50 ms

Tabla 3: Alfas por Transición y por Política

Análisis de Cotas Temporales

Límite Inferior Absoluto (Ley de Amdahl)

Aunque el sistema cuenta con paralelismo, la fase de admisión es estrictamente secuencial: el disparo de T1 requiere consumir el token de exclusión mutua de la plaza P1 (la puerta), lo que fuerza a que cada uno de los 186 clientes atraviesen la transición de forma individual.

Siguiendo la ley de Amdahl (que establece que la mejora máxima en el rendimiento de un sistema está estrictamente limitada por la fracción del programa que no se puede optimizar o paralelizar), el piso temporal queda dado por:

$$t_{min} = N \times d(T_1) = 186 \times 110 \text{ ms} = 20460 \text{ ms} = 20,46 \text{ s}$$

Ninguna combinación de paralelismo o de política va a poder reducir esta cota. La política balanceada y priorizada comparten este piso porque ambas comparten el cuello de botella de la admisión.

Límite Superior Técnico (Escenario Secuencial)

En el peor caso imaginable (conurrencia nula, donde cada cliente recorre el circuito de principio a fin antes de permitir el ingreso del siguiente), el tiempo total sería la suma de los retardos de un invariante completo, multiplicada por la cantidad de clientes que lo transitan. Dado que ambos vendedores se tardan lo mismo en atender según nuestra configuración, la duración queda determinada por si los clientes pasan por confirmación o reserva:

- **Política Balanceada:**

$$\begin{aligned} t_{canc}^{(bal)} &= T1 + T4/T5 + T8 = 60 \text{ ms} + 50 \text{ ms} + 50 \text{ ms} = 160 \text{ ms} \\ t_{conf}^{(bal)} &= T1 + T4/T5 + T9 + T10 = 60 \text{ ms} + 50 \text{ ms} + 50 \text{ ms} + 50 \text{ ms} = 210 \text{ ms} \\ t_{max}^{(bal)} &= t_{canc}^{(bal)} \times 93 + t_{conf}^{(bal)} \times 93 = 160 \text{ ms} \times 93 + 210 \text{ ms} \times 93 = 34410 \text{ ms} = 34,41 \text{ s} \end{aligned}$$

- **Política Priorizada:**

$$\begin{aligned} t_{canc}^{(pri)} &= T1 + T4/T5 + T8 = 60 \text{ ms} + 50 \text{ ms} + 25 \text{ ms} = 135 \text{ ms} \\ t_{conf}^{(pri)} &= T1 + T4/T5 + T9 + T10 = 60 \text{ ms} + 50 \text{ ms} + 50 \text{ ms} + 50 \text{ ms} = 210 \text{ ms} \\ t_{max}^{(pri)} &= t_{canc}^{(pri)} \times 37 + t_{conf}^{(pri)} \times 149 = 135 \text{ ms} \times 37 + 210 \text{ ms} \times 149 = 31610 \text{ ms} = 36,29 \text{ s} \end{aligned}$$

Intervalo de Convergencia

De esta forma, cualquier ejecución real del sistema concurrente debe caer obligatoriamente en los siguientes intervalos cerrados:

$$t_{real}^{(bal)} \in [20,46 \text{ s} ; 34,41 \text{ s}]$$

$$t_{real}^{(pri)} \in [20,46 \text{ s} ; 36,29 \text{ s}]$$

El límite superior de la política priorizada es más alto simplemente porque las dos transiciones de confirmación son más lentas y son las priorizadas.

El piso se alcanza, en teoría, en el **escenario de máxima concurrencia posible**: el disparo simultáneo de T1 junto con T4 y T5 (los dos vendedores atendiendo en paralelo desde la sala de espera P3), mientras en la rama de caja se ejecuta T8 o la secuencia T9+T10, pero nunca ambas a la vez ya que son mutuamente excluyentes por el Recurso Caja.

Análisis Práctico

Para convalidar el modelo analítico, se ejecutó 10 veces al software para ambas políticas.

	T. P. Balanceada [ms]	T. P. Priorizada [ms]
1	21683	21810
2	21811	21805
3	21865	21875
4	21881	21778
5	21841	21866
6	21830	21869
7	21860	21861
8	21883	21823
9	21852	21861
10	21868	21792
Avg	21837.4	21834

Tabla 4: Tiempos Totales de Ejecución por Política

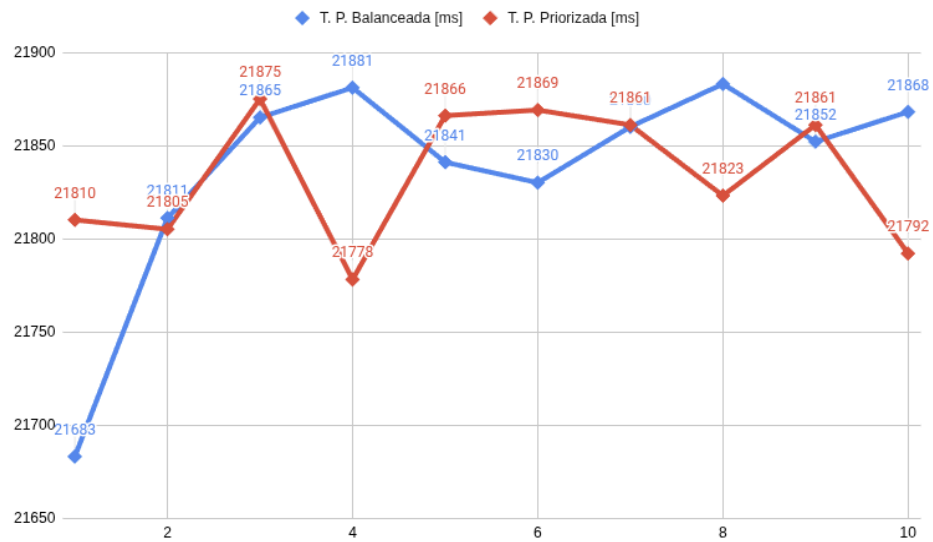


Figura 8: Variación de Tiempos de Ejecución

Dos observaciones saltan a la vista:

1. **Eficiencia respecto a la política:** A pesar de que ambas políticas aplican algoritmos distintos en los conflictos en P3 y P9, el tiempo total difiere en sólo 3.4 ms en promedio, por lo que ambas son igualmente eficientes en términos de tiempo.
2. **Convergencia al límite inferior:** El tiempo real medido (21,837 s y 21,834 s) se ubica apenas ~1,3755 s del piso de Amdahl (20,46 s), lo que representa un overhead de sólo el 6,30% sobre la cota inferior. Esto demuestra que el Monitor explota el paralelismo disponible al máximo.

Análisis de Sensibilidad: ¿Qué Sucede si Cambian los Tiempos?

Como dijimos anteriormente, la elección de los alfas altera la dinámica de competencia en las plazas de conflicto. A continuación se analizan distintas variantes sin tocar la lógica del Monitor ni de la política.

A. Entrada Lenta ($T1 = 200\text{ ms}$)

Hipótesis: Se reemplaza el retardo de 110 ms de DoorTask por 200 ms, manteniendo el resto de los tiempos.

CONFIGURACION		CONFIGURACION	
INITIAL TOKENS = 186		INITIAL TOKENS = 186	
balancedPolicy = true		balancedPolicy = false	
debugEnabled = false		debugEnabled = false	
T1 (entrada recepcion)	= 200 ms	T1 (entrada recepcion)	= 200 ms
T4 (vendedor 2)	= 50 ms	T4 (vendedor 2)	= 50 ms
T5 (vendedor 1)	= 50 ms	T5 (vendedor 1)	= 50 ms
T8 (cancelacion)	= 50 ms	T8 (cancelacion)	= 25 ms
T9 (confirmacion)	= 50 ms	T9 (confirmacion)	= 50 ms
T10 (pago)	= 50 ms	T10 (pago)	= 50 ms
RESULTADOS		RESULTADOS	
Cientes atendidos por el agente en P6:	93 (50.00%)	Cientes atendidos por el agente en P6:	139 (74.73%)
Cientes atendidos por el agente en P7:	93 (50.00%)	Cientes atendidos por el agente en P7:	47 (25.27%)
Reservas confirmadas:	93 (50.00%)	Reservas confirmadas:	149 (80.11%)
Reservas canceladas:	93 (50.00%)	Reservas canceladas:	37 (19.89%)
TIEMPO TOTAL DE EJECUCIÓN: 38569 ms		TIEMPO TOTAL DE EJECUCIÓN: 38576 ms	

Figura 9: Ejecución de Ambas Políticas con $T1$ a 200 ms

B. Entrada Rápida ($T1 = 5\text{ ms}$)

Hipótesis: Se reemplaza el retardo de 110 ms de DoorTask por 5 ms, manteniendo el resto de los tiempos.

CONFIGURACION		CONFIGURACION	
INITIAL TOKENS = 186		INITIAL TOKENS = 186	
balancedPolicy = true		balancedPolicy = false	
debugEnabled = false		debugEnabled = false	
T1 (entrada recepcion)	= 5 ms	T1 (entrada recepcion)	= 5 ms
T4 (vendedor 2)	= 50 ms	T4 (vendedor 2)	= 50 ms
T5 (vendedor 1)	= 50 ms	T5 (vendedor 1)	= 50 ms
T8 (cancelacion)	= 50 ms	T8 (cancelacion)	= 25 ms
T9 (confirmacion)	= 50 ms	T9 (confirmacion)	= 50 ms
T10 (pago)	= 50 ms	T10 (pago)	= 50 ms
RESULTADOS		RESULTADOS	
Cientes atendidos por el agente en P6:	93 (50.00%)	Cientes atendidos por el agente en P6:	94 (50.54%)
Cientes atendidos por el agente en P7:	93 (50.00%)	Cientes atendidos por el agente en P7:	92 (49.46%)
Reservas confirmadas:	93 (50.00%)	Reservas confirmadas:	93 (50.00%)
Reservas canceladas:	93 (50.00%)	Reservas canceladas:	93 (50.00%)
TIEMPO TOTAL DE EJECUCIÓN: 15229 ms		TIEMPO TOTAL DE EJECUCIÓN: 12905 ms	

Figura 10: Ejecución de Ambas Políticas con $T1$ a 5 ms

C. Vendedores Lentos ($T4 = T5 = 200\text{ ms}$)

Hipótesis: Se reemplaza el retardo de 50 ms de Server1Task y Served2Task por 200 ms, manteniendo el resto de los tiempos.

===== CONFIGURACION =====		===== CONFIGURACION =====	
INITIAL_TOKENS = 186		INITIAL_TOKENS = 186	
balancedPolicy = true		balancedPolicy = false	
debugEnabled = false		debugEnabled = false	
T1 (entrada recepcion)	= 110 ms	T1 (entrada recepcion)	= 110 ms
T4 (vendedor 2)	= 200 ms	T4 (vendedor 2)	= 200 ms
T5 (vendedor 1)	= 200 ms	T5 (vendedor 1)	= 200 ms
T8 (cancelacion)	= 50 ms	T8 (cancelacion)	= 25 ms
T9 (confirmacion)	= 50 ms	T9 (confirmacion)	= 50 ms
T10 (pago)	= 50 ms	T10 (pago)	= 50 ms
===== RESULTADOS =====		===== RESULTADOS =====	
Clientes atendidos por el agente en P6: 93 (50.00%)		Clientes atendidos por el agente en P6: 93 (50.00%)	
Clientes atendidos por el agente en P7: 93 (50.00%)		Clientes atendidos por el agente en P7: 93 (50.00%)	
Reservas confirmadas: 93 (50.00%)		Reservas confirmadas: 149 (80.11%)	
Reservas canceladas: 93 (50.00%)		Reservas canceladas: 37 (19.89%)	
TIEMPO TOTAL DE EJECUCIÓN: 22024 ms		TIEMPO TOTAL DE EJECUCIÓN: 21980 ms	
=====		=====	

Figura 11: Ejecución de Ambas Políticas con $T4$ y $T5$ a 200 ms

D. Vendedores Rápidos ($T4 = T5 = 5\text{ ms}$)

Hipótesis: Se reemplaza el retardo de 50 ms de Server1Task y Served2Task por 5 ms, manteniendo el resto de los tiempos.

===== CONFIGURACION =====		===== CONFIGURACION =====	
INITIAL_TOKENS = 186		INITIAL_TOKENS = 186	
balancedPolicy = true		balancedPolicy = false	
debugEnabled = false		debugEnabled = false	
T1 (entrada recepcion)	= 110 ms	T1 (entrada recepcion)	= 110 ms
T4 (vendedor 2)	= 5 ms	T4 (vendedor 2)	= 5 ms
T5 (vendedor 1)	= 5 ms	T5 (vendedor 1)	= 5 ms
T8 (cancelacion)	= 50 ms	T8 (cancelacion)	= 25 ms
T9 (confirmacion)	= 50 ms	T9 (confirmacion)	= 50 ms
T10 (pago)	= 50 ms	T10 (pago)	= 50 ms
===== RESULTADOS =====		===== RESULTADOS =====	
Clientes atendidos por el agente en P6: 93 (50.00%)		Clientes atendidos por el agente en P6: 139 (74.73%)	
Clientes atendidos por el agente en P7: 93 (50.00%)		Clientes atendidos por el agente en P7: 47 (25.27%)	
Reservas confirmadas: 93 (50.00%)		Reservas confirmadas: 149 (80.11%)	
Reservas canceladas: 93 (50.00%)		Reservas canceladas: 37 (19.89%)	
TIEMPO TOTAL DE EJECUCIÓN: 21800 ms		TIEMPO TOTAL DE EJECUCIÓN: 21815 ms	
=====		=====	

Figura 12: Ejecución de Ambas Políticas con $T4$ y $T5$ a 5 ms

E. Vendedores con Tiempos Distintos

Hipótesis: Se reemplaza el retardo de 50 ms de Server1Task por 5ms y Served2Task por 200 ms, y viceversa. El resto de los tiempos se mantiene.

CONFIGURACION		CONFIGURACION	
INITIAL TOKENS = 186		INITIAL TOKENS = 186	
balancedPolicy = true		balancedPolicy = false	
debugEnabled = false		debugEnabled = false	
T1 (entrada recepcion)	= 110 ms	T1 (entrada recepcion)	= 110 ms
T4 (vendedor 2)	= 5 ms	T4 (vendedor 2)	= 5 ms
T5 (vendedor 1)	= 200 ms	T5 (vendedor 1)	= 200 ms
T8 (cancelacion)	= 50 ms	T8 (cancelacion)	= 25 ms
T9 (confirmacion)	= 50 ms	T9 (confirmacion)	= 50 ms
T10 (pago)	= 50 ms	T10 (pago)	= 50 ms
RESULTADOS		RESULTADOS	
Clientes atendidos por el agente en P6:	93 (50.00%)	Clientes atendidos por el agente en P6:	93 (50.00%)
Clientes atendidos por el agente en P7:	93 (50.00%)	Clientes atendidos por el agente en P7:	93 (50.00%)
Reservas confirmadas:	93 (50.00%)	Reservas confirmadas:	93 (50.00%)
Reservas canceladas:	93 (50.00%)	Reservas canceladas:	93 (50.00%)
TIEMPO TOTAL DE EJECUCIÓN: 22004 ms		TIEMPO TOTAL DE EJECUCIÓN: 21850 ms	

Figura 13: Ejecución de Ambas Políticas con T5 a 200 ms y T4 a 5 ms

CONFIGURACION		CONFIGURACION	
INITIAL TOKENS = 186		INITIAL TOKENS = 186	
balancedPolicy = true		balancedPolicy = false	
debugEnabled = false		debugEnabled = false	
T1 (entrada recepcion)	= 110 ms	T1 (entrada recepcion)	= 110 ms
T4 (vendedor 2)	= 200 ms	T4 (vendedor 2)	= 200 ms
T5 (vendedor 1)	= 5 ms	T5 (vendedor 1)	= 5 ms
T8 (cancelacion)	= 50 ms	T8 (cancelacion)	= 25 ms
T9 (confirmacion)	= 50 ms	T9 (confirmacion)	= 50 ms
T10 (pago)	= 50 ms	T10 (pago)	= 50 ms
RESULTADOS		RESULTADOS	
Clientes atendidos por el agente en P6:	93 (50.00%)	Clientes atendidos por el agente en P6:	139 (74.73%)
Clientes atendidos por el agente en P7:	93 (50.00%)	Clientes atendidos por el agente en P7:	47 (25.27%)
Reservas confirmadas:	93 (50.00%)	Reservas confirmadas:	94 (50.54%)
Reservas canceladas:	93 (50.00%)	Reservas canceladas:	92 (49.46%)
TIEMPO TOTAL DE EJECUCIÓN: 21829 ms		TIEMPO TOTAL DE EJECUCIÓN: 21967 ms	

Figura 14: Ejecución de Ambas Políticas con T5 a 5 ms y T4 a 200 ms

F. Confirmación Lenta (T9 = T10 = 200 ms)

Hipótesis: Se reemplazan los retardos de ConfirmationTask por 200 ms para ambas políticas, manteniendo el resto de los tiempos.

CONFIGURACION		CONFIGURACION	
INITIAL TOKENS = 186		INITIAL TOKENS = 186	
balancedPolicy = true		balancedPolicy = false	
debugEnabled = false		debugEnabled = false	
T1 (entrada recepcion)	= 110 ms	T1 (entrada recepcion)	= 110 ms
T4 (vendedor 2)	= 50 ms	T4 (vendedor 2)	= 50 ms
T5 (vendedor 1)	= 50 ms	T5 (vendedor 1)	= 50 ms
T8 (cancelacion)	= 50 ms	T8 (cancelacion)	= 25 ms
T9 (confirmacion)	= 200 ms	T9 (confirmacion)	= 200 ms
T10 (pago)	= 200 ms	T10 (pago)	= 200 ms
RESULTADOS		RESULTADOS	
Clientes atendidos por el agente en P6:	93 (50.00%)	Clientes atendidos por el agente en P6:	139 (74.73%)
Clientes atendidos por el agente en P7:	93 (50.00%)	Clientes atendidos por el agente en P7:	47 (25.27%)
Reservas confirmadas:	93 (50.00%)	Reservas confirmadas:	93 (50.00%)
Reservas canceladas:	93 (50.00%)	Reservas canceladas:	93 (50.00%)
TIEMPO TOTAL DE EJECUCIÓN: 43324 ms		TIEMPO TOTAL DE EJECUCIÓN: 40913 ms	

Figura 15: Ejecución de Ambas Políticas con T9 y T10 a 200 ms

G. Confirmación Rápida ($T9 = T10 = 5\text{ ms}$)

Hipótesis: Se reemplazan los retardos de ConfirmationTask por 5 ms para ambas políticas, manteniendo el resto de los tiempos.

===== CONFIGURACION =====		===== CONFIGURACION =====	
INITIAL_TOKENS = 186		INITIAL_TOKENS = 186	
balancedPolicy = true		balancedPolicy = false	
debugEnabled = false		debugEnabled = false	
T1 (entrada recepcion)	= 110 ms	T1 (entrada recepcion)	= 110 ms
T4 (vendedor 2)	= 50 ms	T4 (vendedor 2)	= 50 ms
T5 (vendedor 1)	= 50 ms	T5 (vendedor 1)	= 50 ms
T8 (cancelacion)	= 50 ms	T8 (cancelacion)	= 25 ms
T9 (confirmacion)	= 5 ms	T9 (confirmacion)	= 5 ms
T10 (pago)	= 5 ms	T10 (pago)	= 5 ms
===== RESULTADOS =====		===== RESULTADOS =====	
Cientes atendidos por el agente en P6:	93 (50.00%)	Cientes atendidos por el agente en P6:	139 (74.73%)
Cientes atendidos por el agente en P7:	93 (50.00%)	Cientes atendidos por el agente en P7:	47 (25.27%)
Reservas confirmadas:	93 (50.00%)	Reservas confirmadas:	149 (80.11%)
Reservas canceladas:	93 (50.00%)	Reservas canceladas:	37 (19.89%)
TIEMPO TOTAL DE EJECUCIÓN: 21711 ms		TIEMPO TOTAL DE EJECUCIÓN: 21700 ms	
=====		=====	

Figura 16: Ejecución de Ambas Políticas con $T9$ y $T10$ a 5 ms

H. Cancelación Lenta ($T8 = 200\text{ ms}$)

Hipótesis: Se reemplazan los retardos de CancellationTask por 200 ms para ambas políticas, manteniendo el resto de los tiempos.

===== CONFIGURACION =====		===== CONFIGURACION =====	
INITIAL_TOKENS = 186		INITIAL_TOKENS = 186	
balancedPolicy = true		balancedPolicy = false	
debugEnabled = false		debugEnabled = false	
T1 (entrada recepcion)	= 110 ms	T1 (entrada recepcion)	= 110 ms
T4 (vendedor 2)	= 50 ms	T4 (vendedor 2)	= 50 ms
T5 (vendedor 1)	= 50 ms	T5 (vendedor 1)	= 50 ms
T8 (cancelacion)	= 200 ms	T8 (cancelacion)	= 200 ms
T9 (confirmacion)	= 50 ms	T9 (confirmacion)	= 50 ms
T10 (pago)	= 50 ms	T10 (pago)	= 50 ms
===== RESULTADOS =====		===== RESULTADOS =====	
Cientes atendidos por el agente en P6:	93 (50.00%)	Cientes atendidos por el agente en P6:	139 (74.73%)
Cientes atendidos por el agente en P7:	93 (50.00%)	Cientes atendidos por el agente en P7:	47 (25.27%)
Reservas confirmadas:	93 (50.00%)	Reservas confirmadas:	93 (50.00%)
Reservas canceladas:	93 (50.00%)	Reservas canceladas:	93 (50.00%)
TIEMPO TOTAL DE EJECUCIÓN: 29306 ms		TIEMPO TOTAL DE EJECUCIÓN: 29265 ms	
=====		=====	

Figura 15: Ejecución de Ambas Políticas con $T8$ a 200 ms

I. Cancelación Rápida ($T8 = 5\text{ ms}$)

Hipótesis: Se reemplazan los retardos de CancellationTask por 5 ms para ambas políticas, manteniendo el resto de los tiempos.

CONFIGURACION	RESULTADOS
INITIAL TOKENS = 186 balancedPolicy = true debugEnabled = false T1 (entrada recepcion) = 110 ms T4 (vendedor 2) = 50 ms T5 (vendedor 1) = 50 ms T8 (cancelacion) = 5 ms T9 (confirmacion) = 50 ms T10 (pago) = 50 ms	Clientes atendidos por el agente en P6: 93 (50.00%) Clientes atendidos por el agente en P7: 93 (50.00%) Reservas confirmadas: 93 (50.00%) Reservas canceladas: 93 (50.00%) TIEMPO TOTAL DE EJECUCIÓN: 21760 ms
INITIAL TOKENS = 186 balancedPolicy = false debugEnabled = false T1 (entrada recepcion) = 110 ms T4 (vendedor 2) = 50 ms T5 (vendedor 1) = 50 ms T8 (cancelacion) = 5 ms T9 (confirmacion) = 50 ms T10 (pago) = 50 ms	Clientes atendidos por el agente en P6: 139 (74.73%) Clientes atendidos por el agente en P7: 47 (25.27%) Reservas confirmadas: 149 (80.11%) Reservas canceladas: 37 (19.89%) TIEMPO TOTAL DE EJECUCIÓN: 21858 ms

Figura 15: Ejecución de Ambas Políticas con $T8$ a 5 ms

Lectura de los Resultados

De los 9 escenarios planteados (A-I), se obtienen un total de 20 ejecuciones (10 con política balanceada y 10 con priorizada). En la política balanceada, las 10 ejecuciones cumplen con el 50/50 en ambos conflictos y lo único que cambia es el tiempo total, mientras que para la política priorizada hay **6 ejecuciones que no cumplen el reparto esperado**:

- **Escenario B - Entrada rápida:** reparto equitativo para ambos conflictos
- **Escenario C - Vendedores Lentos:** el reparto es equitativo para el conflicto de P3 pero se cumple para el conflicto de P9.
- **Escenario E - Vendedor 2 Rápido, Vendedor 1 Lento:** reparto equitativo para ambos conflictos
- **Escenario E' - Vendedor 1 Rápido, Vendedor 2 Lento:** el reparto se cumple para el conflicto en P3 pero es equitativo para el conflicto en P9
- **Escenario F - Confirmación Lenta:** el reparto se cumple para el conflicto en P3 pero es equitativo para el conflicto en P9
- **Escenario H - Cancelación Lenta:** el reparto se cumple para el conflicto en P3 pero es equitativo para el conflicto en P9

Es sabido que la política solamente se activa cuando dos o más transiciones pujan por un token de una misma plaza al mismo tiempo, es decir, cuando se da un conflicto efectivo.

En la mayoría de los escenarios que no cumplen, se observa un patrón: cuando la velocidad de un upstream es relativamente mucho más rápida que el downstream (es decir, los tokens se procesan más lento de lo que llegan), las plazas en conflicto se sobresaturan. Analicemos cada caso:

1. Tomemos como caso inicial de análisis al escenario C:
 - La primera vez que llega un solo token a P3, tanto T2 como T3 se sensibilizan al mismo tiempo, por lo tanto hay conflicto efectivo y se recurre a la política, la cual prioriza la rama del Vendedor 1 esa primera vez, disparándose T2 y haciendo que el vendedor pase a atender al cliente en P5.
 - Como los tokens llegan más rápido de lo que el vendedor 1 termina de atender, el siguiente token va a llegar y solamente se va a sensibilizar T3, por lo que el conflicto efectivo no tiene lugar, la política no va a actuar y T3 se dispara.
 - Seguramente cuando el Vendedor 1 termina, ya va a haber otro token en P3 y el Vendedor 2 va a seguir estando ocupado, así que solamente se sensibiliza T2 y la política tampoco actúa.
 - Este comportamiento se mantiene durante todo el programa hasta que termina, lo que provoca que el reparto sea equitativo entre agentes debido a que el conflicto efectivo nunca se da y la política es incapaz de actuar.
 - Ahora, ¿por qué el reparto si se cumple para el conflicto de P9? Porque el upstream de ese sector es lo suficientemente lento para el downstream, lo que provoca que la saturación de tokens no se de y, por lo tanto, la política puede actuar: por eso el reparto se cumple.
2. Para los escenarios F y H la explicación es parecida. En este caso, aun cuando una sola de las ramas (sea cancelación o confirmación) sea la “rama lenta”, esa lentitud afecta a toda la zona, a diferencia del caso del escenario C donde ambas ramas tienen que ser lentas para afectar a la política de su plaza en conflicto. Esto se debe al recurso compartido en P10: cuando la transición de la rama lenta toma este recurso, toda la zona experimenta dicha lentitud ya que la otra rama no puede avanzar. Esto es lo que provoca la saturación de tokens en P9 y que la política no pueda actuar en esa plaza pero si en P3.
3. El escenario B es un caso muy particular:
 - Incluso con sus tiempos originales, los vendedores producen tokens en P9 relativamente más rápido de lo que se procesan en la zona de resolución de caso. El tiempo original de T1 = 110 ms otorgaba el balance necesario para que no hubiera mucha diferencia relativa de velocidad y que la saturación de tokens en P9 no se de, permitiendo que la política actúe.
 - Por otro lado, los vendedores tienen sobreoferta de tokens como en el escenario C (por lo que la política no puede actuar en P3), la diferencia es que ahora la diferencia relativa de velocidad es causada por la entrada rápida y no por la lentitud forzada de los vendedores.
4. Analicemos el escenario E:
 - Un cliente llega a P3 cada 110 ms.
 - Punto de Vista del Conflicto en P3:
 - *[110 ms de iniciar el programa]*: Llega el **primer token** a P3, se da el conflicto efectivo, la política se “activa” y va hacia la zona del Vendedor 1 por la prioridad. El vendedor se va a desocupar 200 ms después y recién va a poder competir por el token que llegue después de los 310 ms de correr el programa.

- [220 ms]: Llega el **segundo token**. El Vendedor 1 está ocupado, así que el conflicto efectivo no se da y el cliente es atendido obligatoriamente por el Vendedor 2 porque es el único disponible.
 - [225 ms]: Se desocupa el Vendedor 2, el cual no hace nada hasta que llegue el siguiente token a los 330 ms.
 - [310 ms]: Se desocupa el Vendedor 1. **Ambos vendedores quedan desocupados.**
 - [330 ms]: Llega el **tercer token**. Como ambos vendedores están desocupados, el conflicto efectivo se da y la política actúa, enviando al cliente al Vendedor 1.
 - Esto se repite una y otra vez, de tal forma que los tokens se terminan repartiendo de forma equitativa aún cuando el conflicto se da cada un token de por medio. En este caso, no hay saturación de tokens, sino que el problema está en los tiempos elegidos. Si el **vendedor a priorizar es más lento que el ingreso de clientes y el otro vendedor es más rápido, se fuerza a que los tokens se turnen entre ellos de forma equitativa.**
 - Con respecto al reparto equitativo que se da en P9, no supimos encontrar una explicación.
5. En el caso E', el reparto frente al conflicto en P3 si se cumple porque justamente **el vendedor a priorizar es más rápido que el ingreso de clientes**, de tal forma que cuando el conflicto efectivo se da, eventualmente la política si puede contrarrestar la ventaja que tiene el Vendedor 1 por ser más rápido. Nuevamente, no terminamos de entender por qué sucede el reparto equitativo en P9.

Conclusiones:

- La política es reactiva pero no prohibitiva: su única herramienta es redirigir un conflicto que ya existe en el grafo cuando se hace efectivo. Si la semántica temporal destruye el conflicto efectivo o lo hace significativamente menos frecuente, la política queda incapacitada para imponer el umbral.
- Como habíamos analizado anteriormente, el tiempo total de ejecución es muy sensible al retardo en T1, confirmando que es la variable que fija el piso de Amdahl. Esto queda demostrado a partir del Escenario A.

Cumplimiento de Invariantes

Invariantes de Plaza

Para auditar el comportamiento del sistema y certificar el cumplimiento de los invariantes de plaza luego de cada disparo, tal como se solicita en el enunciado, se implementó el método `checkPlaceInvariants()` en la clase `PetriNet`.

```
private boolean checkPlaceInvariants() {
    int[] expectedSums = {1, 5, 1, 1, 1, 5};
    int[] actualSums = {
        marking[1] + marking[2],
        marking[2] + marking[3] + marking[4],
        marking[5] + marking[6],
        marking[7] + marking[8],
        marking[10] + marking[11] + marking[12] + marking[13],
        marking[0]
        + marking[2]
        + marking[3]
        + marking[5]
        + marking[8]
        + marking[9]
        + marking[11]
        + marking[12]
        + marking[13]
        + marking[14]
    };

    for (int i = 0; i < expectedSums.length; i++) {
        if (actualSums[i] != expectedSums[i]) {
            return false;
        }
    }
    return true;
}
```

Figura 16: Verificación de Invariantes de Plaza implementado en el Código

La implementación se basa en una comparación directa entre las sumas esperadas por invariante (ver [ecuaciones de invariantes de plaza](#)), declaradas en el arreglo `expectedSums[]`, y las sumas reales de los tokens presentes en el marcado actual de la red, representadas en el arreglo `actualSums[]`.

Este método es invocado cada vez que se actualiza la Red mediante el método `updatePN(firingVector)`, que a su vez es ejecutado por el Monitor cada vez que un hilo dispara una transición con éxito. Dependiendo del resultado de este método, el sistema responde diferente:

- **Retorna true:** Si el flag `debugEnabled` está activo, se imprime por consola un mensaje gris confirmando el cumplimiento. Este mensaje permite trazar en tiempo de ejecución que la red no ha sido corrompida.
- **Retorna false:** Se lanza una `IllegalStateException` con el marcado actual dado que una violación de los invariantes de plaza es una situación que NUNCA debería ocurrir en una ejecución correcta del sistema, ya que representaría un bug grave. En lugar de continuar la ejecución en un estado corrupto, se aborta el programa.

```

Invariantes de plazas respetados al disparar T2. Marking: [4, 1, 0, 0, 5, 1, 0, 1, 0, 0,
1, 0, 0, 0, 0]
Hilo Agent1 ha disparado T2
Hilo Agent1 ha despertado T5
Agent1 atiende a su cliente [6]
Hilo Served1 no puede disparar T5 porque no está en la ventana de tiempo (quedan 49 ms)
Hilo Door ha adquirido el mutex para T0
Invariantes de plazas respetados al disparar T0. Marking: [3, 0, 1, 0, 4, 1, 0, 1, 0, 0,
1, 0, 0, 0, 0]
Hilo Door ha disparado T0
Entraron 9 personas
Hilo Agent1 ha adquirido el mutex para T2
Hilo Agent1 no puede disparar T2 y se bloquea
Hilo Door ha adquirido el mutex para T1

```

Figura 17: Output del Sistema demostrando la Verificación de Invariantes de Plaza

De esta forma, luego de verificar que un disparo sea posible (sea por marcado o por tiempo), se ejecutan los siguientes pasos en orden estricto:

1. Se calcula el nuevo marcado mediante la ecuación fundamental de la Red.
2. Se verifica que el nuevo marcado cumpla con todos los P-Invariantes.
3. Se recalculan las transiciones sensibilizadas para el próximo ciclo y se guardan sus timestamps de sensibilización.

Invariantes de Transición

Analizar el cumplimiento de los invariantes de transición es más complejo. Para hacerlo, debemos implementar una forma de registrar las transiciones que se disparan durante toda la ejecución y una manera de procesarlas para contabilizar el cumplimiento de dichas invariantes.

CL_Logger fue la clase que implementamos y que registra automáticamente en un archivo de texto plano (transitions.log) la secuencia exacta de transiciones disparadas. Luego este log es procesado por un script de validación en Python (validate_petri_net.py), el cual utiliza expresiones regulares para contabilizar el cumplimiento de los T-Invariantes.

Expresión Regular (RegEx) de la Red

Una Regex es un método formal que define un patrón de búsqueda secuencial sobre cadenas de caracteres. Matemáticamente, están basadas en la teoría de lenguajes formales y autómatas finitos, combinando caracteres literales con metacaracteres (símbolos con funciones operacionales especiales) para describir lenguajes regulares. Se utilizan principalmente para buscar, capturar, validar y reemplazar estructuras de texto complejas de manera automatizada y eficiente.

La verificación del cumplimiento de los Invariantes de Transición plantea un desafío crítico en los sistemas multi-hilo debido al fenómeno del **entrelazado de hilos (Interleaving)**. Cuando el software se ejecuta con alta concurrencia, múltiples hilos disparan transiciones en paralelo de manera asíncrona. Aunque cada cliente individual cumple un ciclo de vida lineal y válido dentro de la Red de Petri, los disparos se registran estrictamente en el orden cronológico en que ocurren. Como consecuencia, el log resultante no es una secuencia ordenada de ciclos aislados, sino una única cadena de texto masiva donde las transiciones de un cliente aparecen

fragmentadas e intercaladas entre las transiciones de otros procesos simultáneos. En este caso, la Regex se va a utilizar para certificar que las secuencias de disparos registradas correspondan estrictamente a ciclos válidos de la red, independientemente del interleaving provocado por la concurrencia.

Para asegurar la robustez de este proceso, se adoptó un flujo de trabajo estructurado en tres fases:

1. **Construcción de la Regex:** En primer lugar, la expresión regular fue diseñada utilizando la plataforma Debuggex^[2]. Esta herramienta permite convertir la sintaxis de la Regex en un diagrama visual. El objetivo principal de esta fase fue mapear los invariantes de tal forma que construyeran a la Red de Petri original.

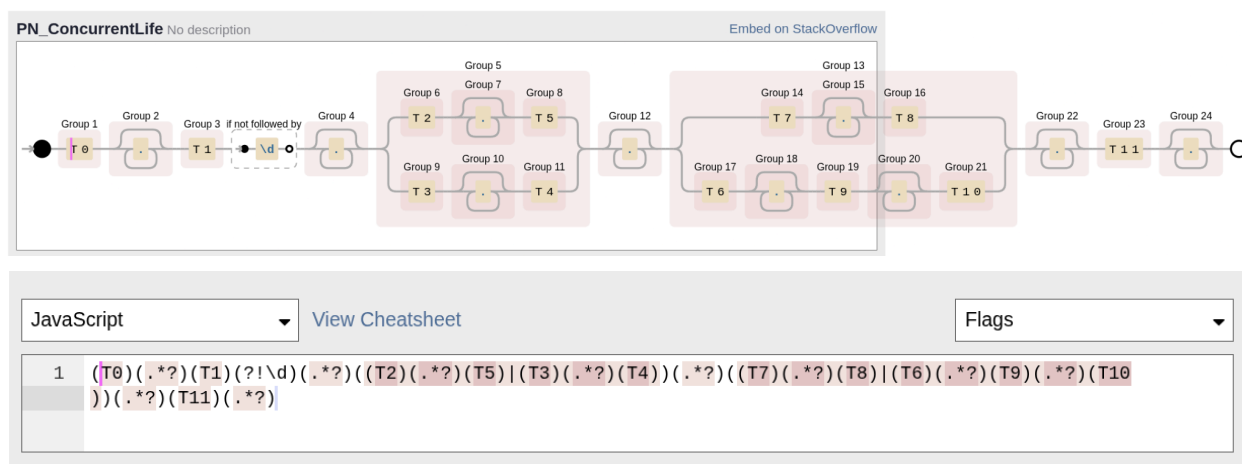


Figura 18: Implementación en Debuggex

Para comprender cómo este patrón emula el flujo transaccional de la agencia de viajes, se detalla a continuación la función técnica de sus metacaracteres y cuantificadores:

- **Cuantificador No Codicioso o Lazy (.*?):** Es el componente fundamental para lidiar con la concurrencia. El punto `.` coincide con cualquier carácter, el asterisco `*` indica cero o más repeticiones, y el signo de interrogación `?` fuerza a que la búsqueda sea "perezosa" (encuentra la coincidencia más corta posible).
 - En el log, los disparos de un cliente específico aparecen mezclados con los de otros procesos. Esta expresión permite capturar y saltar los caracteres de hilos ajenos intercalados en el flujo, permitiendo que la Regex conecte de forma limpia las transiciones consecutivas de un mismo invariante sin romper la secuencia.
- **Operador de Bifurcaciones (|):** Funciona como el operador condicional **OR**. En la expresión regular, se utiliza en dos bloques críticos para representar los conflictos estructurales de libre elección:
 - `((T2)...(T5) | (T3)...(T4))`: Mapea el conflicto en la sala de espera P3. Obliga al patrón a ramificarse para validar de forma equivalente si el proceso avanzó por la línea comercial del Vendedor 1 o del Vendedor 2.

Análisis de los Logs

Se realizó la ejecución del programa para ambas políticas y se procesaron las rutas que tomaron los 186 clientes en los dos casos con el script de python. El script redujo completamente el log a cero en ambos casos, devolviendo estado VÁLIDO.

Caso A: Análisis de la Política Balanceada

```
=====
VALIDACIÓN DE INVARIANTES - RED DE PETRI
=====
Transiciones cargadas (3069 caracteres)
Iteración 1: Encontrados 93 invariantes
Iteración 2: Encontrados 47 invariantes
Iteración 3: Encontrados 46 invariantes
-----
RESULTADOS (Iteraciones: 4)
-----
Estado: ✓ VÁLIDO
Invariantes encontrados: 186
Transiciones procesadas: 3069 / 3069
Invariantes por variante:
  • T0T1T2T5T6T9T10T11: 93 - Variante A (T2-T5-T6-T9-T10 = Served1 → Confirmation)
  • T0T1T2T5T7T8T11: 0 - Variante B (T2-T5-T7-T8 = Served1 → Cancellation)
  • T0T1T3T4T6T9T10T11: 0 - Variante C (T3-T4-T6-T9-T10 = Served2 → Confirmation)
  • T0T1T3T4T7T8T11: 93 - Variante D (T3-T4-T7-T8 = Served2 → Cancellation)
Conteo de transiciones en archivo:
  • T0: 186
  • T1: 186
  • T2: 93
  • T3: 93
  • T4: 93
  • T5: 93
  • T6: 93
  • T7: 93
  • T8: 93
  • T9: 93
  • T10: 93
  • T11: 186
✓ Todas las transiciones fueron procesadas correctamente
=====
```

Figura 20: Procesamiento de las Transiciones Registradas mediante Python - Política Balanceada

El conteo individual de transiciones revela que el Vendedor 1 atendió exactamente a la mitad de la población (T2 con 93 disparos) y el Vendedor 2 a la otra mitad (T3 con 93 disparos). Esto comprueba que el método `resolveP6n7Conflict` interceptó exitosamente el conflicto estructural sobre la sala de espera, distribuyendo los hilos de manera homogénea en base a las métricas de ocupación en tiempo real, priorizando alternadamente al que haya atendido menos.

Un hallazgo crítico es la polarización total de los caminos: **el sistema solo ejecutó las variantes A y D**. No se registró ni un solo disparo de las variantes B o C, aun cuando la rama de cancelación es el doble de rápida que la de confirmación.

Este caso demuestra la estabilidad y robustez que tiene la implementación: **un agente se encarga exclusivamente de las confirmaciones y el otro de las cancelaciones**, balanceando la carga de forma equitativa y eficiente. Aún cuando los alfas son asimétricos para las ramas de confirmación y cancelación, la política logró un reparto entre las dos rutas de salida.

Caso B: Análisis de la Política Priorizada

```

=====
VALIDACIÓN DE INVARIANTES - RED DE PETRI
=====
Transiciones cargadas (3237 caracteres)
Iteración 1: Encontrados 74 invariantes
Iteración 2: Encontrados 56 invariantes
Iteración 3: Encontrados 56 invariantes
-----
RESULTADOS (Iteraciones: 4)
-----
Estado: ✓ VÁLIDO
Invariantes encontrados: 186
Transiciones procesadas: 3237 / 3237
Invariantes por variante:
  • T0T1T2T5T6T9T10T11: 112 - Variante A (T2-T5-T6-T9-T10 = Served1 → Confirmation)
  • T0T1T2T5T7T8T11: 27 - Variante B (T2-T5-T7-T8 = Served1 → Cancellation)
  • T0T1T3T4T6T9T10T11: 37 - Variante C (T3-T4-T6-T9-T10 = Served2 → Confirmation)
  • T0T1T3T4T7T8T11: 10 - Variante D (T3-T4-T7-T8 = Served2 → Cancellation)
Conteo de transiciones en archivo:
  • T0: 186
  • T1: 186
  • T2: 139
  • T3: 47
  • T4: 47
  • T5: 139
  • T6: 149
  • T7: 37
  • T8: 37
  • T9: 149
  • T10: 149
  • T11: 186
✓ Todas las transiciones fueron procesadas correctamente
=====

```

Figura 21: Procesamiento de las Transiciones Registradas mediante Python - Política Priorizada

El conteo bruto de transiciones convalida empíricamente el desbalanceo prioritario exigido por la cátedra para el primer conflicto estructural de la red. La sumatoria de disparos demuestra que:

$$\text{Clientes Vendedor 1} = T2 = \text{Variante A} + \text{Variante B} = 112 + 27 = 139 (74, 37\%)$$

$$\text{Clientes Vendedor 2} = T3 = \text{Variante C} + \text{Variante D} = 37 + 10 = 47 (25, 27\%)$$

Mientras los clientes atendidos por el agente de P6 se mantuvo por debajo del 75% del total acumulado, la política forzó sistemáticamente la transición T2 (Vendedor 1). Solo cuando se acercó o pasó el umbral redirigió el resto al Vendedor 2. Los datos también comprueban que el sistema responde de forma óptima al umbral del 80% de confirmaciones y 20% de cancelaciones.

A diferencia del caso balanceado, aquí las cuatro variantes operaron activamente. La política reactiva tuvo la libertad de interceptar los hilos en cola y cruzó a clientes de ambos vendedores, permitiendo que:

- 112 clientes del Vendedor 1 (80,58% de su clientela) fueran confirmados y 27 (19,42% de su clientela) cancelados
- 37 clientes del Vendedor 2 (78,72% de su clientela) fueran confirmados y 10 (21,28% de su clientela) cancelados.

El entrelazamiento de las cuatro variantes estabilizó las cuotas de negocio (74.73% Vendedor 1, 80.11% confirmaciones) de manera robusta y matemáticamente consistente con los umbrales programados, validando que la política priorizada cumple su función.

CONCLUSIONES

En este trabajo práctico, hemos logrado modelar y validar con éxito un sistema concurrente utilizando Redes de Petri y su posterior implementación en Java. La transición desde el modelo teórico en PIPE hacia un entorno de ejecución real nos permitió profundizar en conceptos fundamentales de concurrencia, sincronización y gestión de recursos.

Las principales conclusiones derivadas del análisis son:

- **Robustez y Vivacidad:** A través del análisis formal, demostramos que el sistema es vivo, acotado y libre de deadlocks. La estructura de la red garantiza que el flujo de procesos sea continuo, sin que ningún estado se convierta en un sumidero.
- **Paralelismo y Eficiencia:** La implementación basada en hilos con responsabilidades delimitadas permitió explotar al máximo el paralelismo permitido por la estructura. Al aplicar el algoritmo formal del paper de cátedra, se evidenció que la red requiere un mínimo de 8 hilos disponibles y que hay una cota máxima absoluta de 5 hilos activos simultáneos en las plazas de acción. El sistema alcanzó un rendimiento óptimo muy cercano al límite teórico de la Ley de Amdahl, con un *overhead* mínimo de procesamiento, lo que confirma la eficiencia del Monitor para gestionar la sincronización de hilos.
- **Efectividad de las Políticas:** La implementación de políticas de gestión (balanceada vs. priorizada) demostró que es posible controlar el comportamiento del sistema sin alterar la estructura fundamental de la Red de Petri. La política priorizada logró alcanzar los objetivos de negocio de manera consistente (74.73% para el Vendedor 1 y 80.11% de confirmaciones).
- **Sensibilidad Temporal:** El análisis de sensibilidad temporal reveló que el comportamiento del sistema es altamente dependiente de los alfa en las transiciones. Observamos que si bien la política tiene capacidad de decisión, esta es reactiva y no prohibitiva; cuando los tiempos de ejecución afectan la tasa de conflictos efectivos, la política pierde su capacidad de intervenir. Esto resalta la importancia de una configuración temporal adecuada.
- **Validación Automática:** La implementación de verificaciones automáticas de invariantes (tanto de plaza como de transición) fue crucial. Mientras que los invariantes de plaza se controlaron en tiempo real, para analizar los logs de transiciones en un entorno de alta concurrencia se usaron Expresiones Regulares (Regex) en Python, superando el desafío que implica el entrelazamiento (interleaving) de hilos.

En conclusión, este proyecto reafirma la eficacia de las Redes de Petri como herramienta de diseño, visualización y evaluación de sistemas concurrentes complejos. El formalismo gráfico y matemático del modelo proporciona una base sólida para resolver problemas de sincronización, superando las limitaciones críticas que presentan enfoques tradicionales.

Asimismo, la separación clara entre la estructura (Red), la mediación centralizada de la sincronización (Monitor) y la lógica de negocio (Política) permitió construir un software modular, escalable y adaptable a cualquier topología solicitada, pudiendo cambiar cada aspecto por separado, sea una nueva Red de Petri u otra lógica de negocio, sin necesidad de modificar o implementar un monitor nuevo. Este diseño demostró una notable versatilidad para operar

eficientemente bajo distintos perfiles de carga, cumpliendo de manera fidedigna con la totalidad de los objetivos planteados por la cátedra.

REFERENCIAS BIBLIOGRÁFICAS

1. Ventre, L., & Micolini, O. (2021). *Algoritmos para determinar cantidad y responsabilidad de hilos en sistemas embebidos modelados con Redes de Petri S 3 PR*. ResearchGate. https://www.researchgate.net/publication/358104149_Algoritmos_para_determinar_cantidad_y_responsabilidad_de_hilos_en_sistemas_embebidos_modelados_con_Red_de_Petri_S_3_PR
2. Debuggex. (2013). *Debuggex: Online visual regex tester* [Aplicación web]. <https://www.debuggex.com/>

ANEXO A: TABLA DE MARCADOS ALCANZABLES (MA_i) PARA LAS PA

	P2	P3	P5	P8	P9	P11	P12	P13	P14	ΣM
MA0	0	0	0	0	0	0	0	0	0	0
MA1	1	0	0	0	0	0	0	0	0	1
MA2	1	0	0	1	0	0	0	0	0	2
MA3	1	0	1	0	0	0	0	0	0	2
MA4	1	0	0	0	0	1	0	0	0	2
MA5	1	0	0	0	0	0	1	0	0	2
MA6	1	0	1	1	0	0	0	0	0	3
MA7	1	0	0	0	0	0	0	1	0	2
MA8	1	0	0	1	0	1	0	0	0	3
MA9	1	0	1	0	0	1	0	0	0	3
MA10	1	0	0	1	0	0	1	0	0	3
MA11	1	0	1	0	0	0	1	0	0	3
MA12	1	1	1	1	0	0	0	0	0	4
MA13	1	0	0	1	0	0	0	1	0	3
MA14	1	0	1	0	0	0	0	1	0	3
MA15	1	0	0	0	1	1	0	0	0	3
MA16	1	0	1	1	0	1	0	0	0	4
MA17	1	0	0	0	1	0	1	0	0	3
MA18	1	0	1	1	0	0	1	0	0	4
MA19	1	2	1	1	0	0	0	0	0	5
MA20	1	0	0	0	1	0	0	1	0	3

MA21	1	0	1	1	0	0	0	1	0	4
MA22	1	0	0	1	1	1	0	0	0	4
MA23	1	0	1	0	1	1	0	0	0	4
MA24	1	1	1	1	0	1	0	0	0	5
MA25	1	0	0	1	1	0	1	0	0	4
MA26	1	0	1	0	1	0	1	0	0	4
MA27	1	1	1	1	0	0	1	0	0	5
MA28	0	3	1	1	0	0	0	0	0	5
MA29	1	0	0	1	1	0	0	1	0	4
MA30	1	0	1	0	1	0	0	1	0	4
MA31	1	1	1	1	0	0	0	1	0	5
MA32	1	0	0	0	2	1	0	0	0	4
MA33	1	0	1	1	1	1	0	0	0	5
MA34	0	2	1	1	0	1	0	0	0	5
MA35	1	0	0	0	2	0	1	0	0	4
MA36	1	0	1	1	1	0	1	0	0	5
MA37	0	2	1	1	0	0	1	0	0	5
MA38	1	0	0	0	2	0	0	1	0	4
MA39	1	0	1	1	1	0	0	1	0	5
MA40	0	2	1	1	0	0	0	1	0	5
MA41	1	0	0	1	2	1	0	0	0	5
MA42	1	0	1	0	2	1	0	0	0	5
MA43	0	1	1	1	1	1	0	0	0	5
MA44	1	0	0	1	2	0	1	0	0	5
MA45	1	0	1	0	2	0	1	0	0	5

MA46	0	1	1	1	1	0	1	0	0	5
MA47	1	0	0	1	2	0	0	1	0	5
MA48	1	0	1	0	2	0	0	1	0	5
MA49	0	1	1	1	1	0	0	1	0	5
MA50	1	0	0	0	3	1	0	0	0	5
MA51	0	0	1	1	2	1	0	0	0	5
MA52	1	0	0	0	3	0	1	0	0	5
MA53	0	0	1	1	2	0	1	0	0	5
MA54	1	0	0	0	3	0	0	1	0	5
MA55	0	0	1	1	2	0	0	1	0	5
MA56	0	0	1	0	3	1	0	0	0	5
MA57	0	0	0	1	3	1	0	0	0	5
MA58	0	0	1	0	3	0	1	0	0	5
MA59	0	0	0	1	3	0	1	0	0	5
MA60	0	0	1	0	3	0	0	1	0	5
MA61	0	0	0	1	3	0	0	1	0	5
MA62	0	0	0	0	4	1	0	0	0	5
MA63	0	0	0	0	4	0	1	0	0	5
MA64	0	0	0	0	4	0	0	1	0	5